

FULL STACK DEVELOPMENT WITH MERN

PROJECT DOCUMENTATION

1. INTRODUCTION

Project Title: CUSTOMER REGISTRY

Team: Prasanthi Konakala

Role

- Full Stack MERN Developer
- Frontend Development
- Backend Development
- Database Design
- API Development
- UI Design
- Testing & Deployment

2. Project Overview

Purpose

The **Customer Registry** is a centralized web-based application developed using the **MERN Stack** to efficiently manage customer information, service requests, and customer interactions. The system provides organizations with a single platform to store, update, and retrieve customer records while simplifying customer support operations.

The application helps businesses streamline customer service processes by maintaining a comprehensive history of customer interactions, support requests, and follow-up activities. Instead of relying on manual registers or spreadsheets, organizations can manage all customer-related information through a secure and user-friendly interface.

The system also provides an interactive dashboard that displays customer statistics and analytical insights, helping organizations identify recurring issues, monitor service performance, and improve decision-making. By maintaining organized customer records and reducing manual effort, the application improves operational efficiency, enhances customer satisfaction, and strengthens long-term customer relationships.

Features

- Customer Registration and Management
- Customer Information Management (CRUD Operations)
- Customer Status Tracking
- Dashboard with Customer Statistics
- Customer Analytics and Reports
- Search and Filter Customer Records
- Responsive User Interface
- RESTful API Integration

- Secure Database Management
- Scalable MERN Architecture

3. Architecture

Frontend

The frontend is developed using **React.js** to provide a modern, responsive, and interactive user interface. React components are used to build reusable pages such as the Dashboard, Customer Management, Analytics, Profile, and Settings. API communication is handled using JavaScript, enabling seamless interaction with the backend services.

Frontend Technologies

- HTML
- CSS
- JavaScript
- React.js

Backend

The backend is developed using **Node.js** and **Express.js**, following a RESTful architecture. It processes client requests, performs business logic, validates data, and communicates with the MongoDB database. The backend exposes APIs for customer management, dashboard statistics, analytics, and other application functionalities.

Backend Technologies

- Node.js
- Express.js

Database

The application uses **MongoDB**, a NoSQL database, to store customer records and application data efficiently.

The database maintains information such as:

- Customer Details
- Customer Status
- Support Records
- Dashboard Statistics
- Analytics Data

MongoDB provides flexible document storage, faster data retrieval, and scalability for future application growth.

4. Setup Instructions

Prerequisites

Before running the project, install the following software:

- Node.js
- MongoDB
- Git

- Visual Studio Code

Installation

Step 1: Install Backend Dependencies

```
cd server
```

```
npm install
```

Step 2: Install Frontend Dependencies

```
cd client
```

```
npm install
```

Step 3: Configure Environment Variables

Create a **.env** file inside the **server** directory.

```
MONGO_URI=mongodb+srv://prasanthikonakala16_db_user:prasanthikonakala16@cluster0.xbsikyv.mongodb.net/?appName=Cluster0
```

```
PORT=5000
```

Create a **.env** file inside the **client** directory.

```
VITE_API_URL=http://localhost:5000/api
```

5. Folder Structure

Client

```
client/
├── public/
├── src/
│   ├── api/
│   ├── assets/
│   ├── components/
│   ├── layouts/
│   ├── pages/
│   ├── App.jsx
│   └── main.jsx
└── package.json
```

Description

- **components/** – Reusable UI components.
- **pages/** – Application pages such as Dashboard and Customers.
- **layouts/** – Common layout components.
- **api/** – API request configuration.
- **assets/** – Images, icons, and styles.

Server

server/

```
|
├── config/
├── controllers/
├── middleware/
├── models/
├── routes/
├── app.js
├── server.js
└── package.json
```

Description

- **config/** – Database configuration.
- **controllers/** – Business logic.
- **middleware/** – Request validation and middleware.
- **models/** – MongoDB schemas.
- **routes/** – REST API routes.

6. Running the Application

Backend

```
cd server
npm start
or
npm run dev
```

Frontend

```
cd client
npm run dev
```

7. API Documentation

Method	Endpoint	Description
GET	/api/customers	Retrieve all customers
GET	/api/customers/:id	Retrieve customer by ID
POST	/api/customers	Add a new customer
PUT	/api/customers/:id	Update customer details
DELETE	/api/customers/:id	Delete customer
GET	/api/dashboard	Retrieve dashboard statistics
GET	/api/analytics	Retrieve analytics data

8. Authentication

Authentication ensures that only authorized users can access the application.

The system validates users before allowing access to protected pages such as Dashboard, Customer Management, and Analytics. Authentication helps secure customer information and prevents unauthorized access to application resources.

9. User Interface

The application consists of the following pages:

- Home Page
- Dashboard
- Customer Management
- Analytics
- Profile
- Settings

10. Testing

The project was tested using the following methods:

- Functional Testing
- CRUD Operation Testing
- API Testing
- UI Testing
- Database Testing
- Responsive Design Testing

11. Screenshots / Demo

Include the following:

- Home Page Screenshot

Modern Customer Care Registry

Manage customers, support tickets and analytics from one powerful dashboard.

[Enter Dashboard →](#)



Customer Management

Add, edit and organize customer information with ease.



Ticket Tracking

Create, assign and resolve customer support tickets quickly.



Analytics

Gain insights using beautiful charts and real-time reports.

1000+

Customers

5000+

Tickets Managed

99.9%

Reliability

Ready to simplify customer support?

[Open Dashboard](#)

- Dashboard Screenshot



Recent Customers

5 Records

	Name	Company	Email
R	Rajesh	TCS	rajesh@gmail.com
R	Rahul	Flipkart	rahul@gmail.com
A	Aakash	TCS	aakash@gmail.com
A	Anjali	ABC Pvt Ltd	anjali@gmail.com
S	Suresh	GHM Pvt Ltd	suresh@gmail.com

Quick Actions

+ Add Customer

View Customers

Refresh Dashboard

Dashboard Summary

Total Customers	16
Pending	8
Resolved	3

- Customer Management Screenshot

**CustomerCare**
REGISTRY

MAIN

Dashboard

Customers

Tickets

Analytics

ACCOUNT

Profile

Settings

Exit Dashboard

Q Search customers, tickets...

July 2026

3

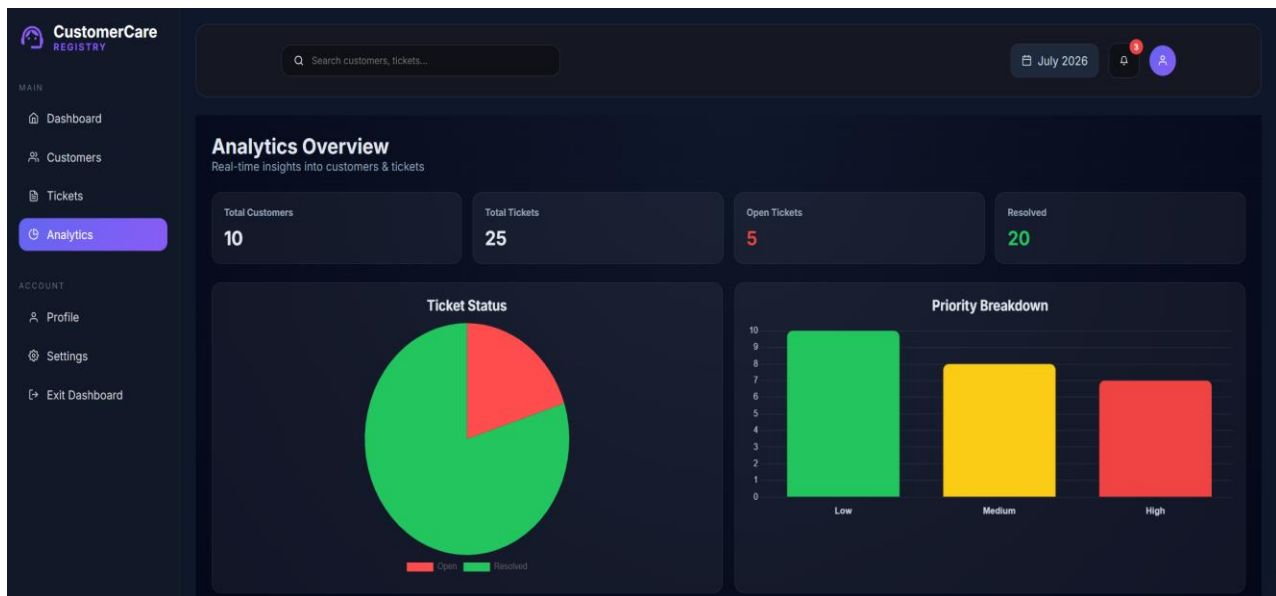
Customers

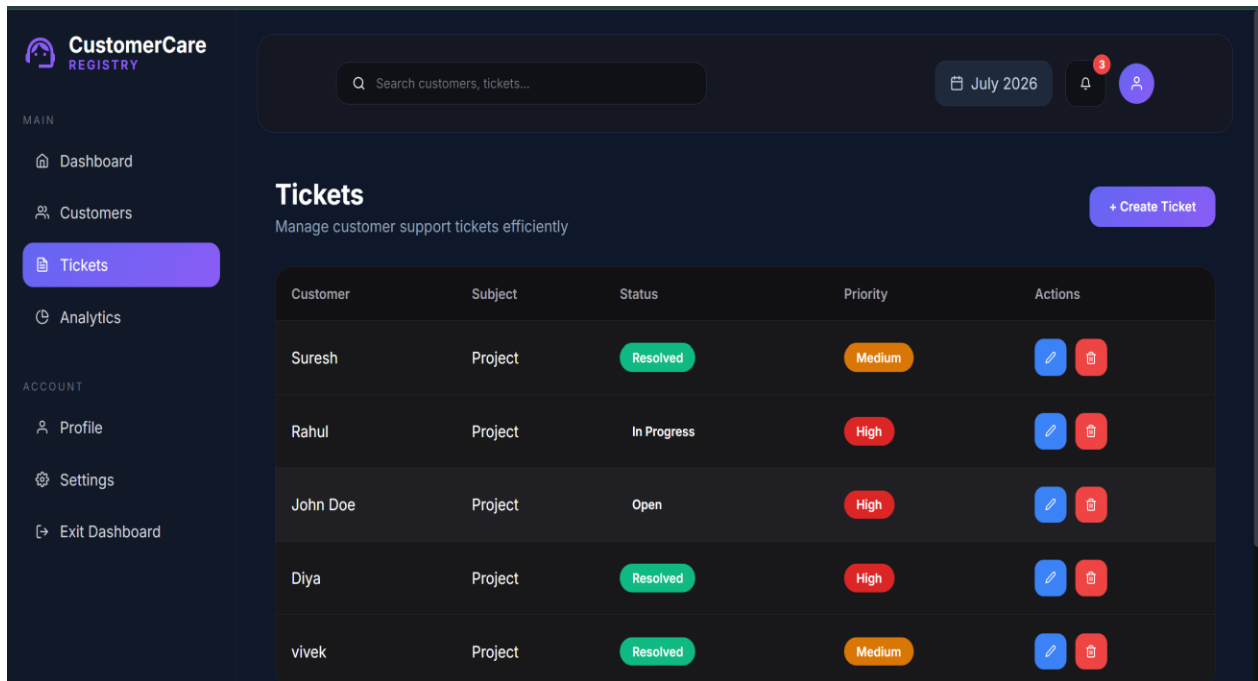
Manage your customer registry.

Search customers...All StatusAll Priority

Customer	Email	Company	Status	Priority	Actions
Rajesh	rajesh@gmail.com	TCS	Active	High	
Rahul	rahul@gmail.com	Flipkart	Active	High	
Aakash	aakash@gmail.com	TCS	Pending	Medium	
Anjali	anjali@gmail.com	ABC Pvt Ltd	Pending	Low	

- Analytics Screenshot





GitHub Repository: <https://github.com/KONAKALAPRASANTHI/customer-registry>

Project Demo Video: https://drive.google.com/drive/folders/1-W_FZzvjiXeFws01p4hvioptElgQeS3L?usp=drive_link

Deployed URL : <https://customer-registry-cl.onrender.com>

12. Known Issues

- Dashboard performance may decrease with a very large dataset.
- Export functionality is not currently available.
- Advanced filtering options can be further improved.

13. Future Enhancements

- Email Notifications
- SMS Notifications
- Report Export (PDF/Excel)
- Advanced Search and Filtering
- Role-Based Access Control
- AI-based Customer Insights
- Mobile Application Support
- Real-time Notifications
- Cloud Deployment Improvements

Skills Required

Technology	Purpose
------------	---------

HyperText Markup Language (HTML)	Used to structure the web pages and application layout.
Cascading Style Sheets (CSS)	Used to design responsive layouts and enhance the visual appearance of the application.
JavaScript (Programming Language)	Used to implement client-side and server-side application logic.
React.js (JavaScript Library)	Used to develop a dynamic, component-based, and responsive user interface.
Node.js (JavaScript Runtime Environment)	Used to execute backend JavaScript and handle server-side operations.
Express.js (JavaScript Framework)	Used to build RESTful APIs, manage routing, and process client requests.
MongoDB	Used as the NoSQL database for storing and managing customer records and application data.